

CosmWasm Contracts

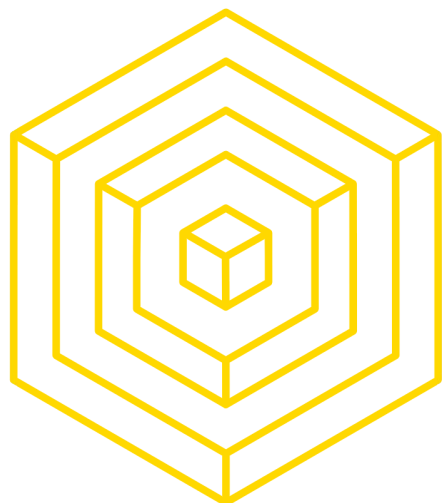


BLOCKCHAIN
AT BERKELEY

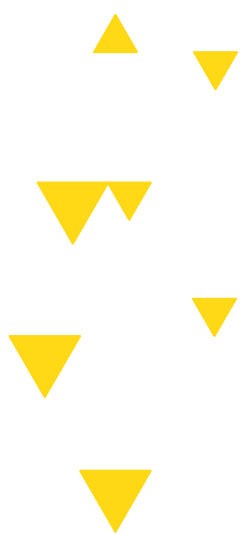


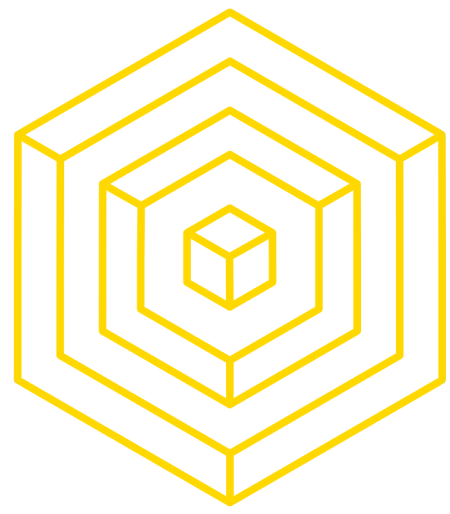
LECTURE OVERVIEW

- 1 ANNOUNCEMENTS
- 2 COSMWASM 2.0
- 3 HW OVERVIEW



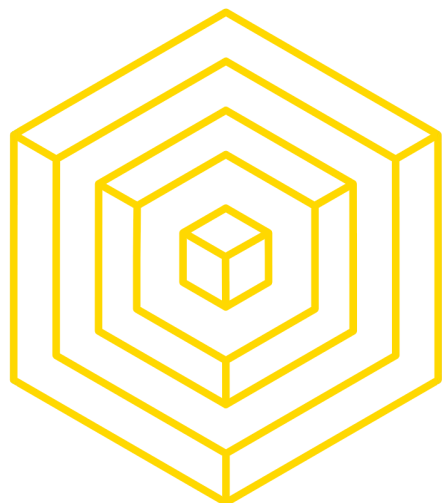
1 Announcements



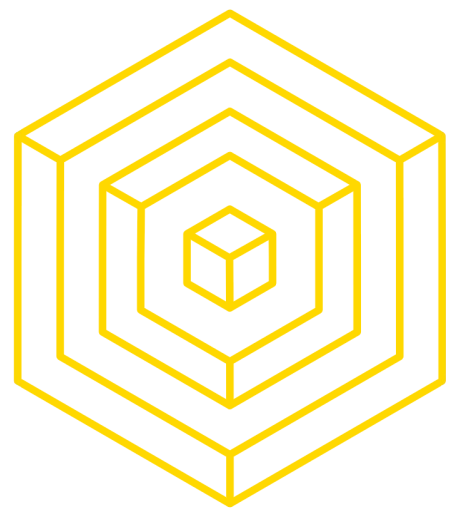


Announcements

- HW6 (Rust) due today @ 11:59PM
- HW 7 (CosmWasm) Released on Github
 - Due in TWO weeks (11/19 @ 11:59PM)



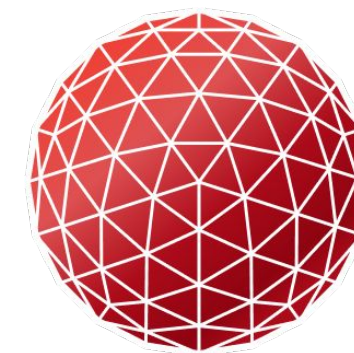
2 CosmWasm 2.0



Soooo, what's Cosmos?

Cosmos

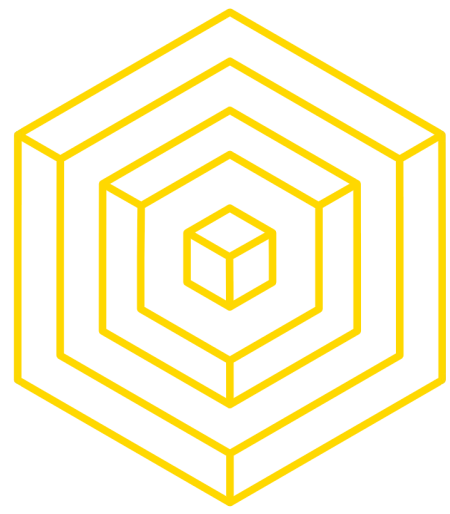
- The Cosmos Network is a decentralized network of independent blockchains
 - Aka “appchains”
 - Before the Cosmos Network, blockchains were siloed and unable to communicate with each other.
 - Examples of appchains below:
 - Osmosis: Primary DEX on Cosmos
 - DEX - DEcentralized eXchange
 - B@B's Old Office!!
 - Mars: Primary lending protocol
 - Stargaze: NFTs and marketplaces
 - Neutron: smart contract platform for DeFi apps
 - Etc.
- ▲ ▼ ■ Each of these are built using the Cosmos SDK, runs its own validators and governance, has its own token and economic model, and can communicate with other appchains through the IBC (Inter-Blockchain Communication)





Important Distinctions/Terms

- Cosmos - ecosystem for interoperable, sovereign, customizable blockchains
 - Not a blockchain itself, but an ecosystem to *host* many independent “appchains” that can talk to each other
- Cosmos SDK - the framework to build a blockchain in the Cosmos ecosystem
 - Provides modules (staking, gov, bank, IBC, wasm) to assemble an appchain
 - Used to build Osmosis, Juno, Stargaze, Injective, etc
- Tendermint (CometBFT) - consensus + networking engine
 - Produces blocks with fast finality and keeps validators in sync
 - Runs under the hood of *most* Cosmos chains



Important Distinctions/Terms (cont.)

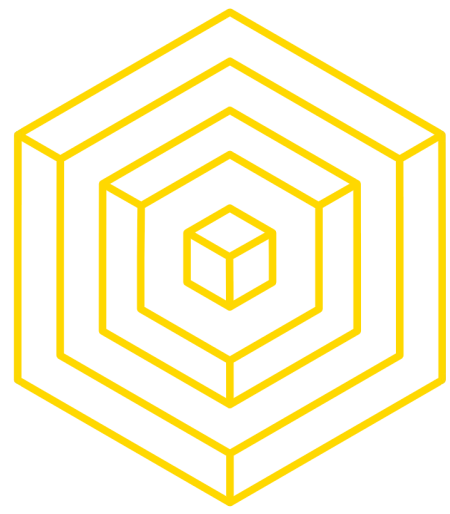
- IBC (Inter-Blockchain Communication Protocol) - Allows for trustless messages and token transfers between different chains
 - Gravity Bridge - Cosmos ↔ Ethereum
 - Axelar Network - Cosmos ↔ Ethereum, Avalanche, Polygon, Bitcoin, etc
 - Wormhole - Cosmos ↔ Solana, Ethereum, Avalanche, BNB, etc
- CosmWasm - Smart contract run time (Wasm VM)
 - Lets chains host programmable contracts compiled to Wasm (usually Rust)
- Rust - language used to create CosmWasm contracts



Now, why is CosmWasm a Big Deal?

CosmWasm's Value

- Host for all smart contracts on the Cosmos network
 - Enables communication and transactions between Cosmos appchains
 - e.g., a DeFi contract on Osmosis can trigger a lending action on Mars Protocol.
- Written in Rust, compiled to WebAssembly (Wasm).
 - This design brings performance, safety, and flexibility, while allowing the same contract to run on any CosmWasm-enabled chain.
- A unified contract layer across Cosmos.
 - Different appchains can host different use cases, but CosmWasm gives them a shared smart contract language; a consistent developer experience across the entire ecosystem



What is Tendermint (aka CometBFT)?

- Tendermint is the **consensus engine** that powers most blockchains in the Cosmos ecosystem
 - Consensus engine - piece of software that makes sure all nodes agree on the same sequence of blocks, even if some nodes go offline or act maliciously!
- What makes it different from PoW/PoS/other consensus mechanisms?
 - Tendermint uses a PoS Byzantine Fault Tolerance (BFT)
 - Meaning it can tolerate up to $\frac{1}{3}$ of nodes being malicious/offline and still keep running!
 - Validators propose and vote on blocks in rounds until $\frac{2}{3}$ agree
 - Tendermint has a networking layer which handles p2p communication between nodes

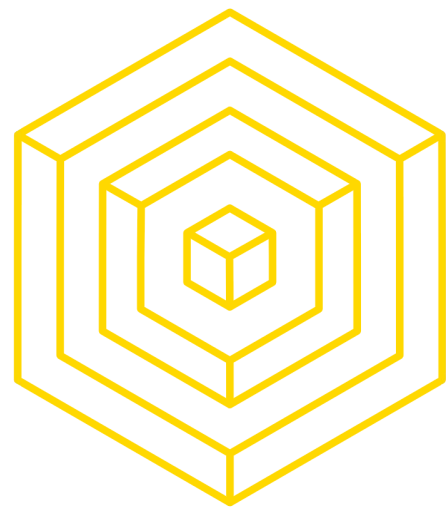
▲ ▼ *** Search up “Tendermint Whitepaper”, download, and read if you’re interested!



CosmWasm 2.0

CosmWasm

- On March 12, Two years after the first stable release of CosmWasm, CosmWasm 2.0 was published.
 - New Fixes Include:
 - Error messages in replies
 - Grpc query
 - Submessage payloads
 - Message responses
 - Optional acknowledgments
 - memo field in `IbcMsg::Transfer`



Error messages in replies

CosmWasm

- Whenever an error is coming from a smart contract, we know it is deterministic.
- In those cases, we now skip redacting the error in wasmd and preserve the full text in the reply (SubMsgResult::Err).
- While this is great for debugging and logging, developers should not make assumptions about the text contents of those errors, which might be changed by the other contract.



Grpc query & Submessage payloads

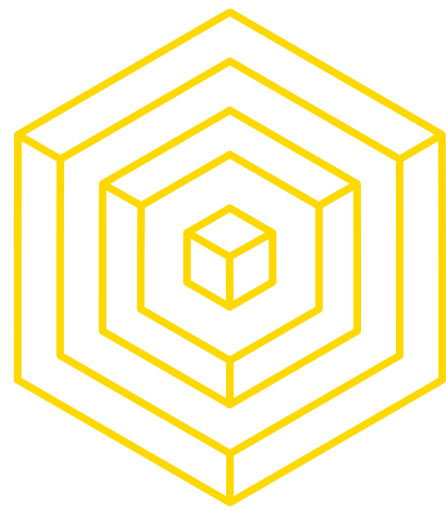
CosmWasm

Grpc query:

The Grpc query is a protobuf-in protobuf-out replacement for Stargate query. This way, responses can be processed more efficiently and use a well-specified encoding. As before, chains have to specify which queries they want to allow.

Submessage payloads:

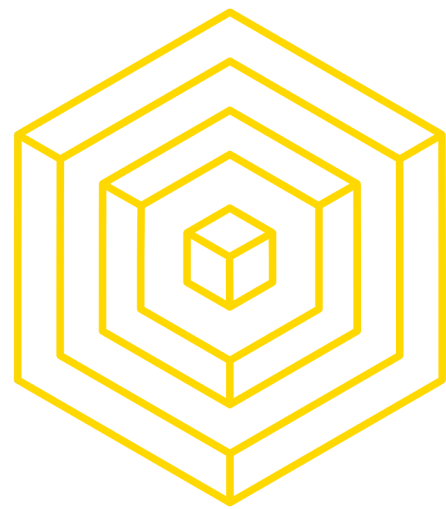
A SubMsg can now get a binary payload that is returned untouched in the reply. This allows you to pass context from the original call to the replay without having to store it to state.



Message responses

CosmWasm

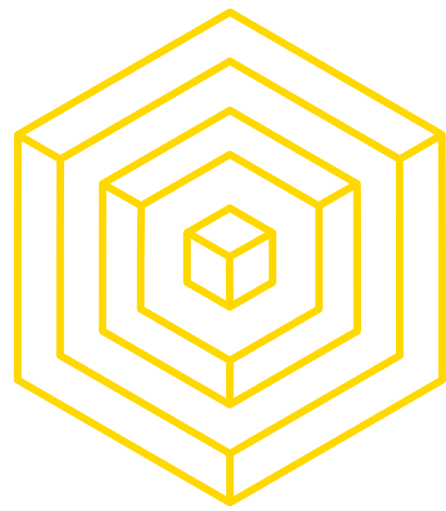
A SubMsgResponse now has msg_responses next to data. In there the message execution results are stored as a list of protobuf Anys. This is needed for getting the data on chains running Cosmos SDK 0.50.



Things that were removed

CosmWasm

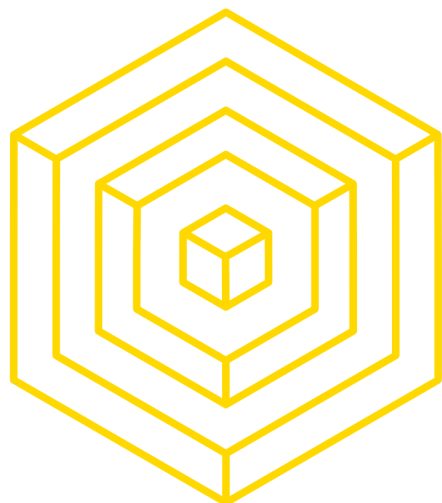
- Time to say goodbye to things not relevant anymore today:
 - The cosmwasml-storage crate. Use cw-storage-plus for types storage access or cosmwasml-std for raw access.
 - Addr and String/&str cannot be compared directly anymore for safety reasons. It's recommended to convert both sides to an Addr first.
 - Mixed type arithmetics were removed (especially multiply Decimal with Uint*). The functions `Uint{128,256}::mul_floor` or `Uint{128,256}::mul_ceil` can be used instead.



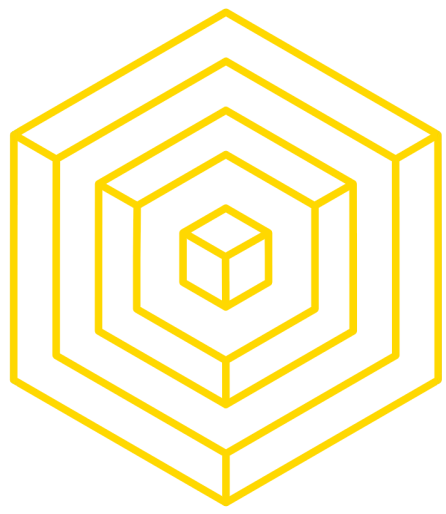
Error messages in replies

CosmWasm

- Whenever an error is coming from a smart contract, we know it is deterministic.
- In those cases, we now skip redacting the error in wasmd and preserve the full text in the reply (SubMsgResult::Err).
- While this is great for debugging and logging, developers should not make assumptions about the text contents of those errors, which might be changed by the other contract.



3 Rust Quirks



Quirks

Control flow

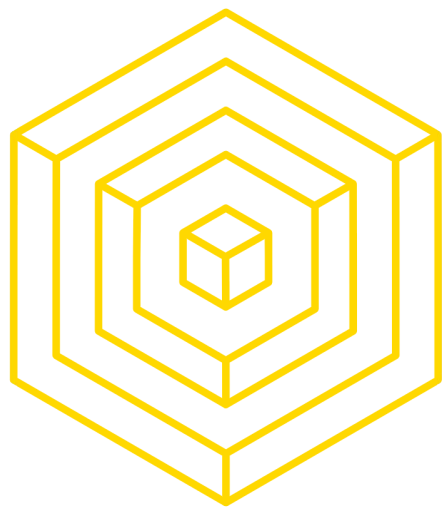
18

Like with `if let`, there is a `while let` variant which repeatedly tests a value against a pattern:

```
1 fn main() {  
2     let v = vec![10, 20, 30];  
3     let mut iter = v.into_iter();  
4  
5     while let Some(x) = iter.next() {  
6         println!("x: {x}");  
7     }  
8 }
```

Here the iterator returned by `v.into_iter()` will return a `Option<i32>` on every call to `next()`. It returns `Some(x)` until it is done, after which it will return `None`. The `while let` lets us keep iterating through all items.

Source: <https://google.github.io/comprehensive-rust/control-flow/novel.html>



Quirks

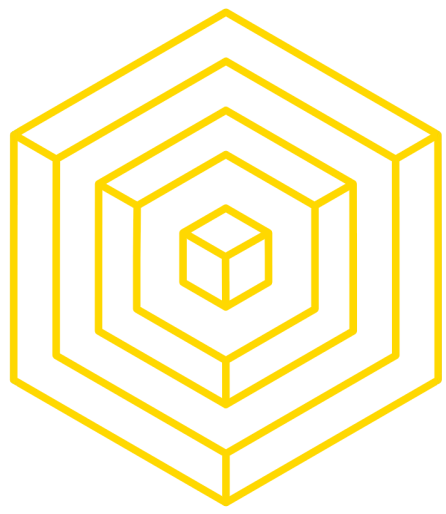
Control flow

The `match` keyword is used to match a value against one or more patterns. In that sense, it works like a series of `if let` expressions:

```
1 fn main() {  
2     match std::env::args().next().as_deref() {  
3         Some("cat") => println!("Will do cat things"),  
4         Some("ls")  => println!("Will ls some files"),  
5         Some("mv")  => println!("Let's move some files"),  
6         Some("rm")  => println!("Uh, dangerous!"),  
7         None       => println!("Hmm, no program name?"),  
8         _          => println!("Unknown program name!"),  
9     }  
10 }
```

Like `if let`, each match arm must have the same type. The type is the last expression of the block, if any. In the example above, the type is `()`.

Source: <https://google.github.io/comprehensive-rust/control-flow/novel.html>



Quirks

Control flow

If you already have variables with the right names, then you can create the struct using a shorthand:

```
1  #[derive(Debug)]
2  struct Person {
3      name: String,
4      age: u8,
5  }
6
7  impl Person {
8      fn new(name: String, age: u8) -> Person {
9          Person { name, age }
10     }
11 }
12
13 fn main() {
14     let peter = Person::new(String::from("Peter"), 27);
15     println!("{peter:?}");
16 }
```

<https://google.github.io/comprehensive-rust/structs/field-shorthand.html>



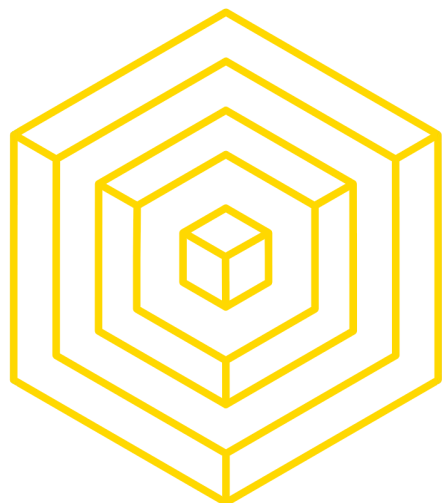
Quirks

Control flow

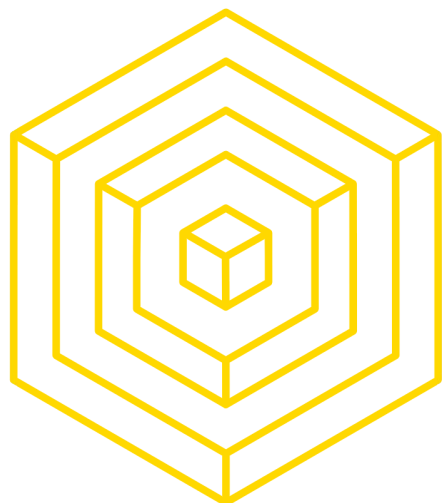
Like C and C++, Rust has support for custom structs:

```
1 struct Person {  
2     name: String,  
3     age: u8,  
4 }  
5  
6 fn main() {  
7     let mut peter = Person {  
8         name: String::from("Peter"),  
9         age: 27,  
10    };  
11    println!("{}", peter.name, peter.age);  
12  
13    peter.age = 28;  
14    println!("{}", peter.name, peter.age);  
15  
16    let jackie = Person {  
17        name: String::from("Jackie"),  
18        ..peter  
19    };  
20    println!("{}", jackie.name, jackie.age);  
21 }
```

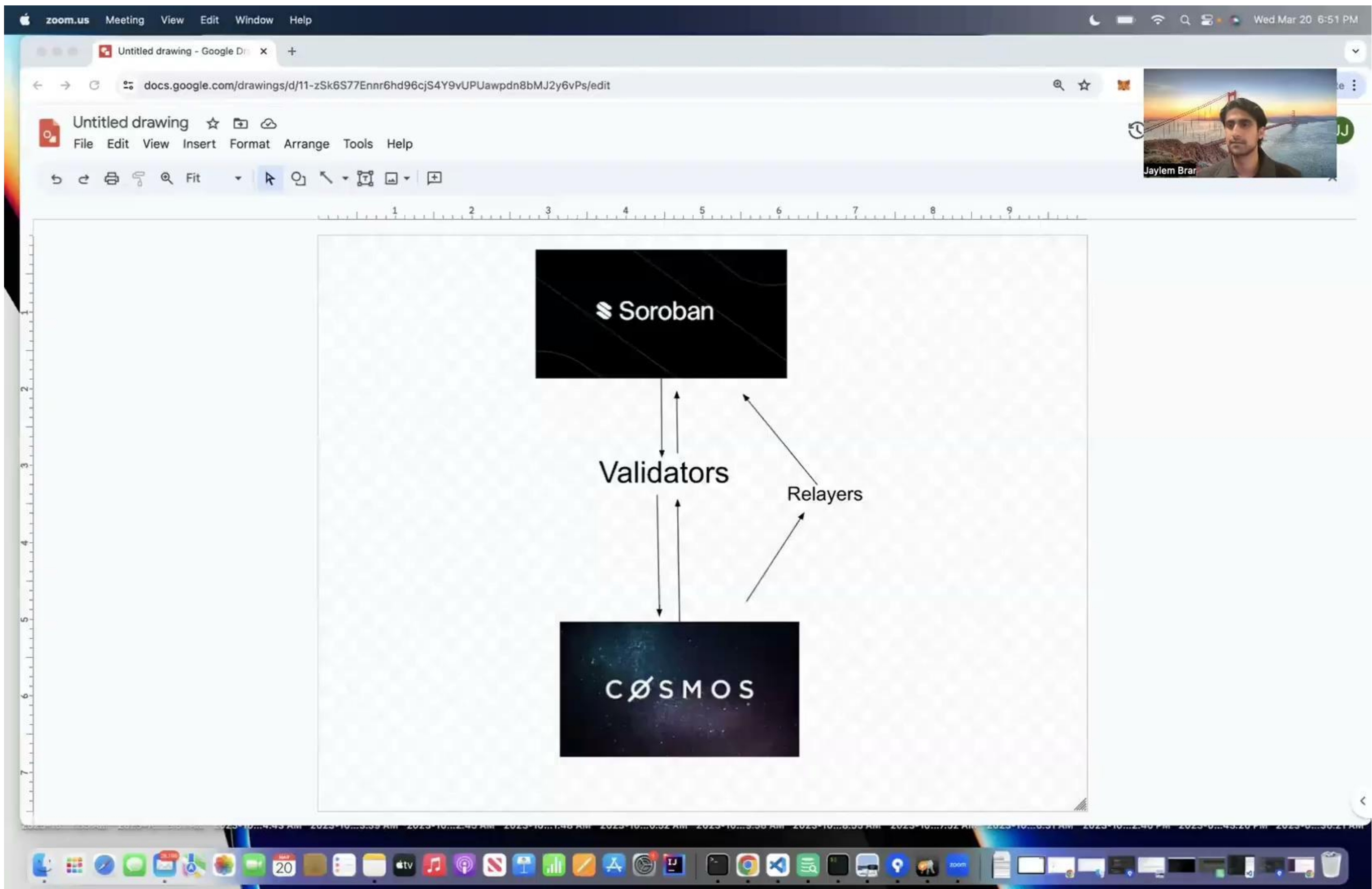
Source: <https://google.github.io/comprehensive-rust/control-flow/novel.html>

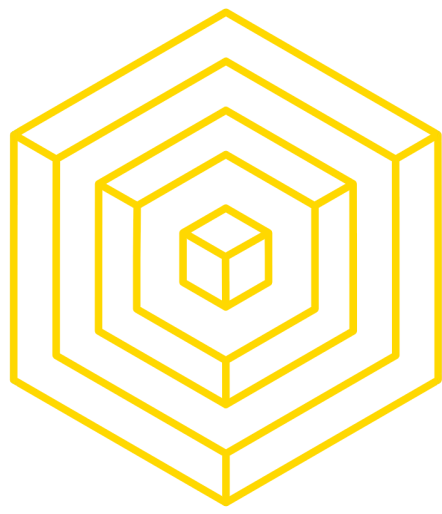


4 JJ Demo

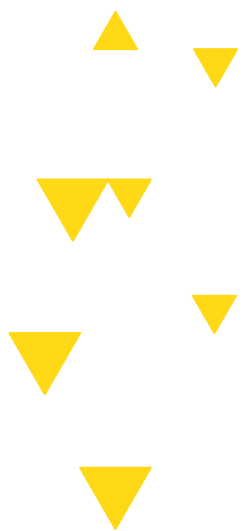


Rust Demo





3 HW OVERVIEW





Installation

25

```
rustup target add wasm32-unknown-unknown
```



Optionally if you want to try out your contracts on a testnet, you will need a [wasmd](#) binary. We would focus on testing contracts with Rust unit testing utility throughout the book, so it is not required to follow. However, seeing the product working in a real-world environment may be nice.

To install `wasmd`, first install the [golang](#). Then clone the `wasmd` and install it:

```
$ git clone git@github.com:CosmWasm/wasmd.git
$ cd ./wasmd
$ make install
```



An additional helpful tool for building smart contracts is the `cosmwasm-check` [utility](#). It allows you to check if the wasm binary is a proper smart contract ready to upload into the blockchain. You can install it using cargo:

```
$ cargo install cosmwasm-check
```



If the installation succeeds, you should be able to execute the utility from your command line.

```
$ cosmwasm-check --version
Contract checking 1.2.3
```



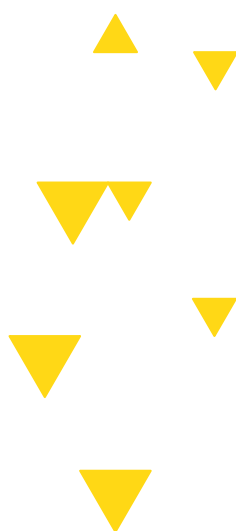


Exercise 1 - Hello World

26

Using the println macro, add code to print the contents of hello_msg.

Using the println macro, add code to print the contents of species_msg





Exercise 2 - Contract Instantiation

- You can think of smart contract code as the blueprint for the contract itself. It does not truly become a useful part of a blockchain until it has been instantiated. Once it is established on the blockchain it can be interacted with.
- In this exercise and the next few we're going to work on the instantiate entry point (as well as two others).
- In Rust (and other C/C++ and Java-like languages), there is the concept of an "entry point". This is typically called main and is the first function called when a compiled application is executed. When compiling a traditional Rust app, the compiler expects there to be a main function from which the rest of the application will start and run.
- There is no main function in CosmWasm - so how does it know instantiate is an entry point (the other two notwithstanding)?
- We instruct the code of that fact with a function attribute, using `entry_point` from the `cosmwasm_std` library:
- Instantiate is one of three such entry points into a smart contract. It's used during deployment to initialize settings for a contract that's been stored on-chain, and create an address on the blockchain where the contract can be called. Maybe you recognize this pattern, as it's called a Constructor in many programming languages.



Exercise 2 - Contract Instantiation

28

- Let's create an empty instantiate function. Here are the steps we're interested in:
 - Write a use declaration to get access to the entry_point attribute from the CW library.
 - Write the signature for a public Rust function called instantiate. For now, we only want to write the function with an empty parameters list, and empty body and no return, like this `pub fn my_function() {}`. We'll fill in the rest as we go along.



Exercise 3 - Returned Value of Instantiate

- The instantiate entry point returns a Rust Result, taking a CosmWasm Response struct and an error type.
- The Response struct returns a few items that allow the contract to communicate back to the caller. It has several types in it for sending messages and data back.
- CosmWasm is based on the Actor Model design pattern. In this pattern, Actors do not talk directly to one another (i.e., do not call functions directly) but rather send messages to one another. Here's a basic interface for the Actor model:

```
pub trait Actor {  
    fn handle(msgPayload: &[u8]) -> Vec<Msg>;  
}  
  
pub struct Msg {  
    pub destination: Vec<u8>,  
    pub payload: Vec<u8>,  
}
```



What is the Actor Model?





COSM WASM Response Model

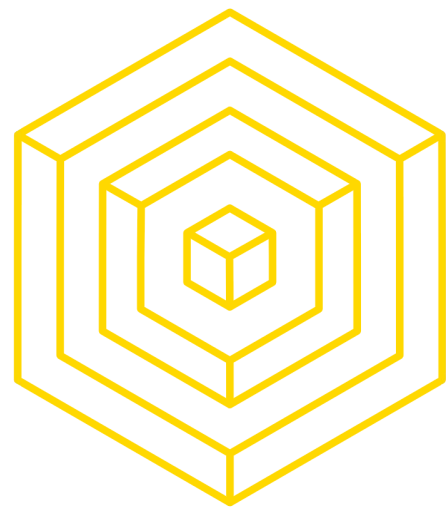
```
#[non_exhaustive]
pub struct Response<T = Empty> {
    pub messages: Vec<SubMsg<T>>,
    pub attributes: Vec<Attribute>,
    pub events: Vec<Event>,
    pub data: Option<Binary>,
}
```




Exercise 3 - Returned Value of Instantiate

For this exercise, we're simply going to elaborate on the instantiate outline we've already created.

1. Add a `Result<R, E>` return to the function. "R" will be `Response` and "E" will be `ContractError`, which we can import as a Rust Crate.
2. Have the function return `Ok(Response::default())`.



Exercise 4 - Entry Point Parameters

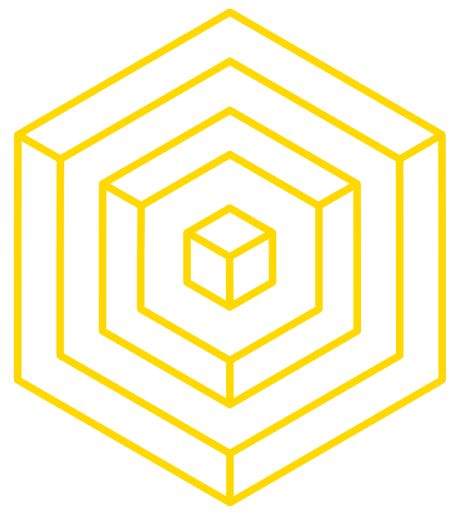
Our instantiate function takes four parameters: `DepsMut`, `Env`, `MessageInfo` and `InstantiateMsg`.

The first three parameters are from the `cosmwasm_std` library, and the last one, `InstantiateMsg`, is a custom struct we will write ourselves.

Deps, short for `_dependencies_`, provides access to CW utilities such as persistent storage; which, as you might have deduced, allows you to read from storage. For writing to storage you'll want the mutable version of this package, called `DepsMut`.

Env, short for `_environment_`, provides information about the block and the transaction the message was executed in.

MessageInfo, contains the sender's (that is, the caller of the contract method) address and funds sent along with the transaction.



Exercise 5 - Storing State

While a smart contract can store information on the blockchain, that comes at a cost in the form of gas and fees, but this ability to store and compute on open data, that everybody can access, is what allows it to be decentralized.

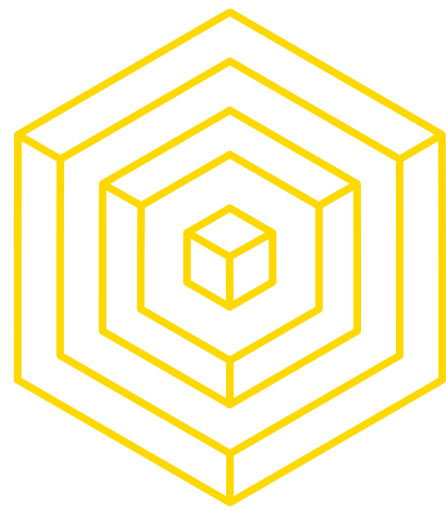
In this exercise, we'll create a State struct to store our basic information when the contract is initialized.

This is typically done in a file named **state.rs** in your CosmWasm project.

Serialize and Deserialize

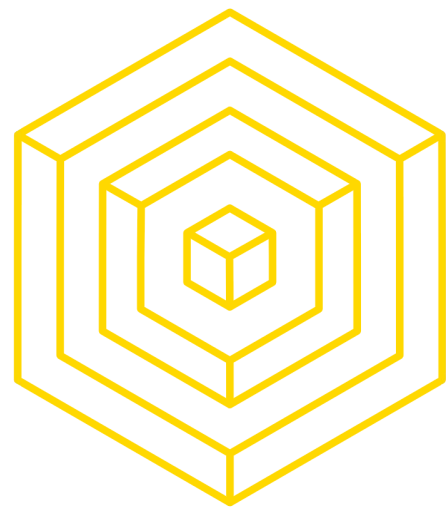
We'll be making use of the [Serde](#) framework to serialize and deserialize the Rust data structures. In our case, we need our upcoming State struct to store data in such a way that it's prepared for storing on the blockchain. From Serde, **Serialize** and **Deserialize** are traits that need to be applied to the struct using the derive attribute.

But first, let's get the struct written out. We'll build the structure here, then talk about the specifics in later exercises.



Exercise 5 - Storing State

- From Serde, Serialize and Deserialize are traits that need to be applied to the State struct. They will be applied using the derive attribute.
- Apply `#[derive(Trait1, Trait2)]` to Serialize and Deserialize.
- Immediately below derive, create a public struct called State.
- State will have four public members called owner, planet_name, planet_sapient and minimum_sapience.
- The members will be of the following types, respectively: Addr, String, Vec<Sapient>, and SapienceScale.



Exercise 6 - Storing the Struct

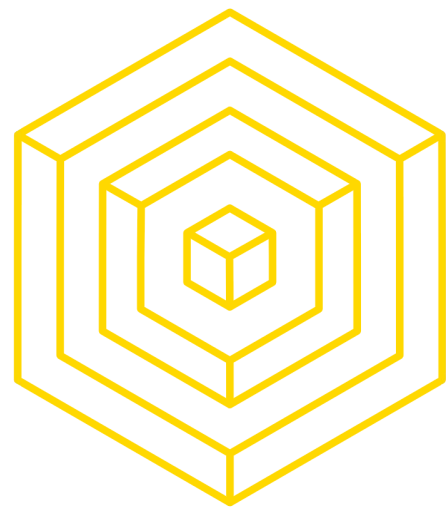
Now that we have our struct to work with, let's begin creating the tools to read and write to it.

CosmWasm Storage

One of the parameters passed to instantiate is `deps: DepsMut`; it contains a member called `storage`:

`Storage`. This is what we will be updating on the blockchain. We'll do it using the **Singleton** implementation from the `cosmwasm_storage` library.

We'll have two separate functions — a write function, called `config`, and a read-only function called `config_read`. Conveniently, `cosmwasm_storage` provides two matching versions called `Singleton` and `ReadonlySingleton`.

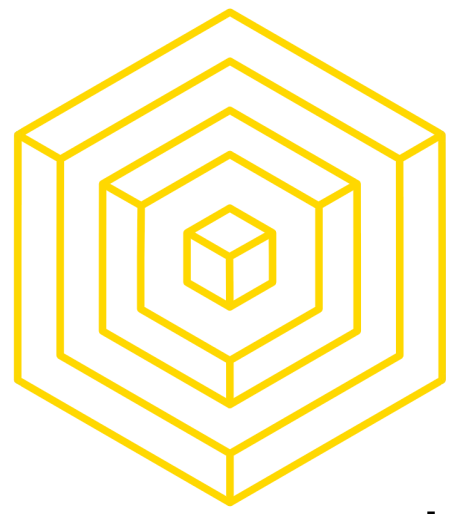


Exercise 6 - Storing the Struct

We're only going to write the signatures for this; we'll add the functionality in the next exercise.

Write two public functions called `config` and `config_read`.

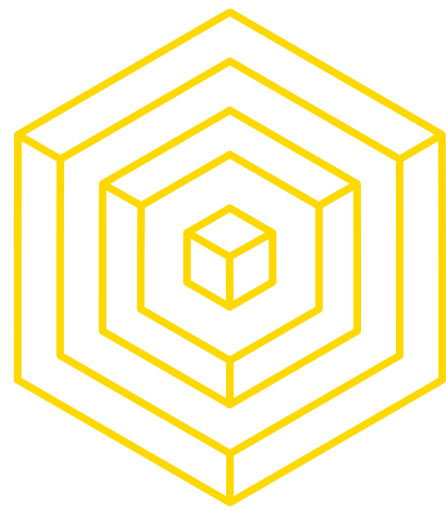
1. `config` will take one argument: `storage: &mut dyn Storage` and return `Singleton<State>`.
2. `config_read` will also take one argument: `storage: &dyn Storage` and return `ReadOnlySingleton<State>`.



Exercise 7 - Singleton and Storage

The `cosmwasm_storage` package provides the singleton method which gives us access to typical storage methods, in particular, load and save, which we will use here.

The singleton function requires a unique key (key: `&[u8]`) as well, to ensure there are no collisions when saving the data. This lets us have multiple types of storage. The key indicates where we will read or save data, and will be the same for both instances of our config functions (i.e. `config` and `config_read`).



Exercise 7 - Singleton and Storage

1. Write a static constant called `CONFIG_KEY` of type `&[u8]`. Assign the value `b"config"` to it.
2. In `config`, return `singleton`, with storage for the first argument, and the key we assigned in step 1, for the second argument.
3. For `config_read`, you'll use `singleton_read` and its arguments will be the same as the parameters for `config`.



Exercise 8 -

Now that we've written our config, let's see how it's applied.

First, let's talk about the instantiate info parameter of type MessageInfo. Right now, MessageInfo is a fairly simple struct:

```
pub struct MessageInfo {  
    pub sender: Addr,  
    pub funds: Vec<Coin>,  
}
```

It's typical for the entity accessing the smart contract to be called the "sender". The sender can provide funds to the contract, which is retained to be accessed and used by the contract.



Exercise 8 -

Every entity that wants to interact with a blockchain needs a unique address. In the case of CosmWasm, addresses in Rust come in the type **Addr**, Addr is a string type that conforms to some cryptographic requirements of the blockchain and can be validated. You will see it often throughout these exercises.

Saving with Config

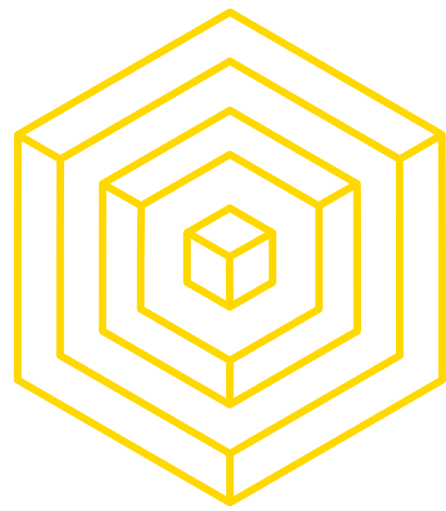
As a reminder, our config function takes a variable of type Storage. This happens to come from the deps argument.

Once config is called, we can save the state by applying the .save() function, and passing in a reference to state (e.g., save(&state)). You can check the state.rs file to remind yourself how config works.

The save function returns a StdResult, so use the ? Rust operator to capture any errors.

For this exercise, input the owner argument and save it to the contract's state.

1. Create a variable called state and initialize it with a State struct that contains a single member, owner.
2. owner will be populated by the info.sender argument.
3. Use config to save &state into deps.storage.



Exercise 9 - Instantiate Msg

We've talked about DepsMut and MessageInfo already; we're going to ignore Env for this lesson, leaving us with InstantiateMsg, which is a structure we'll write to ensure anyone calling our contract complies to our design.

All CosmWasm entry points take either an enum or a struct. For instantiate, structs are typically used to initialize a smart contract with starting values. We need to apply Serialize and Deserialize since this information is coming from outside the contract.

For our Jump Ring smart contract, we're only going to expect a few basics items:

The planet's name, as a String;

A list of sapient creatures on the planet as a Vec<Sapient>;

The minimum sapience allowed for passage through this particular Jump Ring (based on the Interchain endorsed and totally legit SapienceScale, which we can view in species.rs along with Sapient).

There will be additional messages for the other two entry points coming later. We'll do this work in a new file called msg.rs.

1. The "message" for instantiate is a public struct with public members.
2. Write a struct called InstantiateMsg.
3. The three members will be, in order, planet_name, planet_sapient, and minimum_sapience.
4. Apply the correct types to each member.



Attendance/Vitamin

